DM587 - Git for Windows

Use PowerShell - a copy-and-paste guide for any assignment

You will use Git to download the assignment files, save a revision of your work and send it to your private GitLab project. Use the private project assigned to you by the instructor for all assignments.

The routine is: download, write and save, submit, read feedback. If a command reports an error, stop and use the help page. You do not need to memorise these commands.

Before you start

- The instructor must have assigned you a private project on SDU GitLab.

- Git for Windows and Python 3 must be installed. Open the Start menu, search for **PowerShell**, and open it. In Windows Terminal, choose a **PowerShell** tab. Use these commands in PowerShell throughout this guide.

- Check Git by running `git --version`. If it is not installed, use the official Git installation page or ask for help: https://git-scm.com/install/windows.

After installing Git, close PowerShell and open it again so it can find Git. Check Python with `py -3 --version`. If either check fails, use the help page.

`$HOME` automatically means your Windows user folder, usually `C:\Users\your-name`. Leave `$HOME` as written in the commands; you do not replace it with your username.

Your working folders

We strongly recommend this folder layout on your own computer so that you can easily find each assignment:

```
dm587/
  assignments/
    assignment-0/     working copy for Assignment 0
    assignment-1/     working copy for Assignment 1
    assignment-2/     working copy for Assignment 2
    assignment-3/     working copy for Assignment 3
    assignment-4/     working copy for Assignment 4
    assignment-5/     working copy for Assignment 5
    assignment-6/     working copy for Assignment 6
    assignment-7/     working copy for Assignment 7
```

To create all eight folders automatically on your local machine, copy and run this whole block in PowerShell:

```
0..7 | ForEach-Object {
    $assignmentFolder = "$HOME\dm587\assignments\assignment-$_"
    New-Item -ItemType Directory -Force -Path $assignmentFolder | Out-Null
}
```

This creates empty folders in your home folder and keeps any existing files. It does not download assignments. In Stage 1, clone into the empty folder for the assignment you are starting.

After cloning, each `assignment-i` is a separate local copy of **the same private course project**. The clone includes other assignment folders already in the course repository. This is expected. Edit only the files for the assignment you are working on.

The folder commands below use `assignment-1` as an example. Replace its number with the assignment you are starting, from 0 to 7. Use the same private project URL. The folder number is for organising your computer; keep all instructor-provided folder and file names inside the clone unchanged.

Stage 1 - Download the assignment

Start after the instructor confirms that your assignment is ready in your private project. Download it once using the steps below. **If you already downloaded it into this assignment's folder, skip this stage and continue to Stage 2.**

Download once into the assignment folder

Creating the empty folders on page 1 does not download the files. You still need to clone once into the folder for the assignment you are starting.

Open your private project at https://gitlab.sdu.dk. Select **Code > Clone with HTTPS** and copy the URL. It must be your private project URL, rather than the course group URL or your profile URL.

Replace `YOUR_PROJECT_HTTPS_URL` below with the URL you copied. Keep the quotation marks. Then copy and run:

```
New-Item -ItemType Directory -Force -Path "$HOME\dm587\assignments" | Out-Null
cd "$HOME\dm587\assignments"
git clone "YOUR_PROJECT_HTTPS_URL" assignment-1
cd assignment-1
```

Git downloads the files into the empty `assignment-1` folder, creating it if needed. Keep using this same folder to write your solution and submit corrections. **Do not clone again each time you work or submit.** If the folder already contains your downloaded assignment, open it and continue to Stage 2. Do not delete existing work to make cloning work.

You do not need to download updates as part of the normal workflow. If the instructor changes the assignment, they will tell you exactly what to do.

If Git asks you to log in

Enter your **GitLab username**. At the password prompt, use your **personal access token with `write_repository` scope**. You can reuse your working course token. If you need a token, create one in your GitLab personal access token settings and keep its secret value privately. You do not need an API token for this student workflow.

The password field may show nothing while you paste; paste once and press Enter. Do not put your token into the clone URL, Python files, screenshots or messages to the instructor. Git may reuse saved credentials on later commands.

One-time Git identity setup

If you have not already set up Git on this computer, replace the two quoted placeholders with your own details, then run:

```
git config --global user.name "YOUR FULL NAME"
git config --global user.email "YOUR SDU EMAIL ADDRESS"
```

These details label your commits. They are not a password and do not log you in to GitLab. You normally do this only once on your own computer.

Success: your terminal is now in `assignment-1`, and the assignment files are on your computer. Continue to Stage 2.

Stage 2 - Write and save your Python code

Keep this folder as your project root

Open `dm587\assignments\assignment-1` in your code editor. Open the editor's terminal in that same folder. This is the **project root**: the top folder of this local clone, containing `.gitlab-ci.yml` and the instructor's assignment folders.

Each local clone contains the instructor-provided files for the course. Its layout is:

```
$HOME\dm587\assignments\assignment-1\   PROJECT ROOT
  .gitlab-ci.yml                 leave unchanged
  [supplied assignment folder]/
    [supplied Python files]     edit as the assignment sheet directs
  ... other course files
```

The bracketed labels describe the layout; they are not names to type or create. `assignment-1` is your local working-folder name. The required source-folder and file names are listed in each assignment sheet. Keep both levels. Do not move all Python files into the project root or rename the supplied folders.

Work on the files inside this clone

Read the assignment sheet, edit the required Python files and **save them in your editor**. Keep the other course files unchanged. Your code must return the results required by the assignment; you do not submit screenshots or a ZIP of your code.

To check where your terminal is, run:

```
Get-Location
```

The displayed path should end in `dm587\assignments\assignment-1`. You can return to that root from any folder with:

```
cd "$HOME\dm587\assignments\assignment-1"
```

Run the local Python checks given in the assignment sheet, from this project root. Use its exact filenames and commands. This general Git guide does not prescribe an assignment's Python tests.

`py -3` runs Python 3 on Windows. A local check helps you find mistakes; the online grader may test additional inputs.

Saving a file or running Python does not send your work to GitLab. When you want online feedback, continue to Stage 3.

Stage 3 - Submit your saved work

Run these commands in the project root: `dm587\assignments\assignment-1`**.** If needed, open a terminal and first run:

```
cd "$HOME\dm587\assignments\assignment-1"
```

Save your files in your editor, then check what has changed:

```
git status
```

It should say you are on branch `main` and list the solution files you edited. If files you did not intend to change or delete appear, stop and check them before submitting.

Copy and paste these three commands

For an assignment where you edit the supplied files, submit by running:

```
git add -u
git commit -m "Submit assignment"
git push origin main
```

Command	What it does
`git add -u`	Selects changes to files already supplied in your clone.
`git commit`	Saves those selected changes as a local revision.
`git push`	Sends the revision to GitLab and starts automatic grading.

You may use the same commit message each time. You do not have to clone again, create a branch or create a new GitLab project for a resubmission.

`git add -u` also selects deletions of tracked files, so keep the supplied files in place. It does not select new files. If an assignment explicitly asks you to create a new file, run `git add` with that filename before committing, following the assignment sheet. If required files appear under **Untracked files** unexpectedly, ask the instructor to check your folder layout.

Check that the push succeeded

Open your private project in the browser, select **main**, and open the assignment's source files. Confirm that you can see your latest code. The latest commit should show the message you just used.

A successful push submits your revision. A commit saved only on your computer is not a submission. If Git reports an error, use the help page before trying unrelated commands.

Once your code is visible on GitLab, continue to Stage 4.

Stage 4 - Read feedback and improve your code

Where to look on GitLab

- Open your private project in the browser.
- Open **Build > Pipelines**.
- Select the pipeline matching your latest pushed commit.
- Open the grading job named in the assignment sheet.
- Wait until it finishes, then read the job log for your grade and failed checks.

Pushing normally starts the grading automatically once the instructor has enabled it for the assignment. You do not need to run a grading job manually after each push. If the assignment template or grading job is not available yet, contact the instructor.

Read the numerical grade and its maximum in the job log. For example, a result out of 100 may look like:

```
Grade: 80.00/100.00
```

The feedback may name a failed test, give points deducted, and include an error or expected/actual result. A job marked **Failed** can still have a valid numerical grade because a check failed. That status alone is not a decision about passing the course.

The exact feedback format and available points depend on the assignment. If the job says **Score unavailable** or **grading setup error**, contact the instructor. Any downloadable feedback locations are given in the assignment sheet.

To get feedback on a correction

Edit your local solution, save it and run its local checks. Return to the project root and repeat the same three submission commands:

```
git add -u
git commit -m "Submit assignment"
git push origin main
```

Then open the **new** pipeline for that push. An old pipeline still describes its old revision; it does not change when you edit a file on your computer. Local Python output is shown in your terminal, while the online grade is shown in GitLab.

Before the deadline

Successfully push your work to **main** before the deadline stated in the assignment sheet. Scores from late pushes are excluded. Follow the assignment sheet's scoring policy. A timely push remains on time if grading runs later because the runner was queued.

Your normal cycle: edit, save, check locally, run the three commands, read the new feedback.

Help - only if a command reports a problem

The folder already exists when cloning

Do not delete it. If it already contains your downloaded assignment, open it and continue to Stage 2. If you are unsure, ask for help so you do not lose unfinished work. Do not repeatedly clone into the same nonempty folder.

Not a git repository

Your terminal is probably outside the clone. Return to your working root:

```
cd "$HOME\dm587\assignments\assignment-1"
git status
```

Nothing to commit

Save your files in the editor and run `git status`. If your branch is ahead of `origin/main`, your work is already committed: run `git push origin main`. If Git says everything is up to date, check your code and latest pipeline online. If the edited files are missing from both places, check that your editor has the correct working copy open.

Push rejected: fetch first or non-fast-forward

GitLab contains changes that are not in this local copy. Stop and send the error message to the instructor, who will give you the appropriate commands. Keep your saved work. Do not force-push, discard files or clone again to make the error disappear.

Git or Python is not recognised

After installing Git for Windows, close PowerShell and open it again, then retry `git --version`. If `py -3 --version` is not recognised, try `python --version`. If that reports Python 3, use `python` instead of `py -3` for the Python checks. Ask for help if neither Python command works.

Git asks who you are

Use the one-time name/email setup in Stage 1, then repeat the commit and push commands. Your Git identity and your GitLab login are separate.

Authentication failed, access denied or no pipeline

Check your private project URL and GitLab username. Use your personal access token at the HTTPS password prompt. Contact the instructor if access fails, you cannot push to main or a successful push has no grading job.

For help, send the error message, project URL and relevant pipeline/job URL. Hide passwords and tokens in messages and screenshots.